

CyberAgentFlow

What a record has to hold before a failure can be named, and the five failures the first record named

The page opens by refusing the metric everything else in this area reports.

“Most evaluations of agentic cyber tools ask whether the task got done. That tells you almost nothing about how it got done, or what it cost.”

website, CyberAgentFlow, opening

Everything below is vocabulary for the second sentence. The project has three parts and each one supplies its own terms. There is a record, which fixes what counts as an entry. There is a stack, which is local on purpose and which decides what the record can see. And there is one engagement read back afterwards, which produced five named failures and one number.

“Reaching the pivot took roughly 50 minutes. With the identified waste removed, the estimated time is roughly 11 minutes.”

website, CyberAgentFlow, What it found

The agent reached the objective. Four fifths of the clock went to nothing. No success rate can hold both of those facts, which is the argument the project exists to make.

Contents

1	Active	2
1.1	The record	2
1.2	The stack	4
1.3	The engagement	5
1.4	The five failure patterns	7
1.5	The reading	9
2	Working	10
3	Peripheral	11
4	The disagreements, listed	12
5	What crosses into the shared lexicon	12
6	Sources	13

1 Active

1.1 The record

CyberAgentFlow

Definition: An instrumentation framework that records the whole interaction lifecycle of an agent-driven cyber engagement as a structured execution trace, and an analysis pass that reads the archived trace afterwards for inefficiency rather than for outcome.

Distinguishes it from: An agent. CyberAgentFlow does not decide anything during the engagement. It is the layer that makes the deciding visible, and the agent it was pointed at is an ordinary language model driving ordinary tools. Also from a benchmark: a benchmark supplies targets and scores runs, and this supplies neither.

Operational test: Remove the framework and ask what the engagement leaves behind. If the answer is a terminal history and a verdict, the framework was doing the work.

In my project: The name has four forms in my own files. The page file is `cyberagenttrace.md`, the heading on it is CyberAgentFlow, the body of the page writes Cyber-Agent-Flow, and `SUMMARY.md` writes Cyber AgentFlow. The published abstract adds a fifth thing, which is not a spelling but a scope: the framework is presented there as AgentFlow, general to agent-driven workflows, and cybersecurity is the domain it is aimed at. CyberAgentFlow is the project. The paper title is Cyber-Agent-Flow. AgentFlow is the framework without the domain.

execution trace

“Sessions are archived as structured execution traces, and an analysis model inspects them afterwards for inefficiency.”

website, CyberAgentFlow, What gets recorded

Definition: The stored record of one engagement, in which every prompt, model response, tool invocation, command output, analyst annotation and timestamp is kept in sequence and kept together, so the engagement can be replayed rather than summarised.

Distinguishes it from: Three thinner records, each of which is derivable from it and none of which can be turned back into it. A transcript keeps what was said and drops what the tools returned. A tool log keeps what the tools returned and drops what the model was reasoning about when it asked. A benchmark result keeps only whether the run ended in success.

Operational test: Take any command in the record and ask two questions: what prompt produced it, and what its output changed about the next step. If either answer requires guessing, the record is a log.

In my project: The cluster overview states the requirement this satisfies before this project is named: the trace has to be detailed enough to say the failure was in the structure rather than merely that the team lost. That sentence is about a team. Here the same requirement is met for one agent driving real tools, and the thing a failure gets attributed to is a wasted call rather than a misrouted approval.

interaction lifecycle

“prompts, model responses, tool invocations, command outputs, analyst annotations, and timing”

website, CyberAgentFlow, What gets recorded

Definition: The span of an engagement from the objective being issued to the objective being reached or abandoned, treated as one recordable object with six kinds of entry.

Distinguishes it from: A turn and a step, which are units inside it. Lifecycle is the right word because it commits to recording the two things that have no turn boundary: the annotation a person writes after watching something go wrong, and the wall clock that keeps running while the model is deciding.

Operational test: Count the entry kinds actually present in a stored session. If model responses are there and command outputs are not, or the other way round, the lifecycle is half recorded and every failure pattern below becomes invisible.

In my project: Six kinds. Five of them come from the machine. One, the annotation, comes from the person watching, which is the only place a human enters the record at all.

analyst annotation

Definition: A note written into the trace by the person supervising the engagement while it runs, entered as a first-class entry with the same standing as a command output.

Distinguishes it from: A code assigned afterwards by a researcher reading the transcript. The annotation is written by the person who was there, at the moment, with the state of the engagement in front of them, and it lands inside the record rather than beside it.

Operational test: Ask when it was written and by whom. During the run and by the supervisor makes it an annotation. Afterwards and by an analyst who was not present makes it a code, and codes belong to a different study design.

In my project: It is the entry kind the agent cannot produce, and the reason a trace can carry why a step looked wrong at the time rather than only that it turned out wrong.

context budget

“a context budget that summarises older history while keeping recent exchanges intact”

website, CyberAgentFlow, What gets recorded

Definition: The rule governing what an agent carries forward once an engagement outruns the model’s window: recent exchanges kept whole, older history compressed.

Distinguishes it from: A context window, which is a capacity rather than a policy. Two agents on the same model can hold the same window and spend it differently, and only the spending is a design decision.

Operational test: Run the engagement past the window and ask what is still retrievable. If an early reconnaissance result survives in some form, there is a budget. If it is simply gone, there is only a window.

In my project: It is the first suspect for the first of the five failure patterns. A command re-issued with unchanged parameters is the signature of a result that was summarised away, and that is an architecture fault rather than a reasoning fault.

1.2 The stack

local stack

“The stack runs locally.”

website, CyberAgentFlow, What gets recorded

“cloud-hosted agents send prompts and network telemetry off-site, which is not acceptable on a real engagement”

website, CyberAgentFlow, What gets recorded

Definition: The deployment constraint that every component touching engagement data, which is the model, the gateway, the tool servers and the trace store, runs on hardware the engagement’s owner controls.

Distinguishes it from: A data-handling policy. A policy is a promise about what a third party does with material it holds. The local stack is the arrangement under which the third party never holds it.

Operational test: List the outbound connections opened during an engagement. If any of them carries a prompt or a command output, the stack is not local, and the word for what left is network telemetry belonging to somebody else.

In my project: The page states this as a deliberate choice rather than a constraint imposed from outside, and it is the choice that fixes most of the rest of the architecture. A hosted model would have made the gateway unnecessary and the framework unusable.

Model Context Protocol interface

Definition: The declared connector through which a tool is exposed to a language model as a callable capability, with its arguments and its return typed, so the model invokes the tool instead of composing a command line and hoping.

Distinguishes it from: A shell. A shell accepts anything and reports failure as free text. An interface accepts a declared argument set and returns a structured result, which is what turns a tool invocation into an entry with fields rather than a line in a terminal.

Operational test: Ask whether the record can say which tool was called with which arguments without parsing a string. If it can, the tool is behind an interface.

In my project: Kali utilities sit behind an MCP server, nmap and Metasploit among them. Two of the five failure patterns are about what happens inside a tool that carries its own interpreter, which is the seam the protocol does not reach across, and that is a design finding rather than a complaint about the model.

gateway

Definition: The single point every model call passes through, doing routing and access control, so which model answered and what it was permitted to see are properties of the record rather than of whichever component happened to call.

Distinguishes it from: A client library, which every component would otherwise carry its own copy of, with its own credentials and its own idea of which model is current.

Operational test: Change the model for one engagement. If that is a configuration change and not a code change, there is a gateway.

In my project: LiteLLM routes and controls access; Ollama hosts the model itself. The pairing is what makes a private model as easy to swap as a hosted one, which is the practical condition for the local stack being adopted rather than admired.

tool permission and IP allowlist

Definition: The two constraints the analyst sets on an engagement: which tools the agent may invoke at all, and which addresses it may reach with them.

Distinguishes it from: An approval, which is per-action and arrives after the agent has proposed something. A permission is per-capability and set in advance, and the agent never sees the action it was not allowed to consider.

Operational test: Ask whether the agent can propose the forbidden thing. Under a permission it cannot. Under an approval it can, and is refused, and the refusal is data.

In my project: This is the authority mechanism this project has, and it is coarser than the one CHART has on purpose. There is no queue, no denial and no timeout to record, only a boundary the agent is never offered. That is the right design for a live engagement against a real target and the wrong one for studying how a person decides, which is why the two projects do not share an authority vocabulary.

the analyst in the loop

“issuing objectives in natural language, watching actions as they happen, annotating throughout, and constraining what the agent may touch through tool permissions and IP allowlists”

website, CyberAgentFlow, What gets recorded

Definition: The supervisory role this project gives a person: set the objective, set the boundary, watch, and write into the record. Four verbs, none of which is approve.

Distinguishes it from: The teammate role in the rest of the cluster, where the person holds a veto on named actions and the veto is the manipulation. Here the person holds the boundary and the objective, and inside those the agent runs without asking.

Operational test: Take the person out of the room mid-engagement. If the agent stops, the person was a gate. If it keeps going within the allowlist, the person was a supervisor.

In my project: It is the honest position for an engagement where an agent acts faster than a human can review and the actions are against a real host. The cluster’s second challenge says the authority structure is a safety requirement in cyber defense. This project’s answer to that requirement is a boundary set before the fact, not an approval routed during it.

1.3 The engagement

engagement

Definition: One authorized exercise against a real target under a stated objective, from the objective being issued to the objective being reached or given up.

Distinguishes it from: An episode in a simulator, which resets. An engagement does not reset. The target keeps whatever state the run left it in, and the time is spent whether or not the step worked.

Operational test: Ask what happens on a second run. If the target is where the first run left it, it was an engagement, and the arithmetic of waste is arithmetic about real minutes.

In my project: This is why the timing result is worth reporting from a single case. In a simulator, fifty minutes is a hyperparameter. On an engagement, it is fifty minutes.

pivot

“The agent starts on an externally reachable host, works through reconnaissance and service identification, exploits a vulnerable web server, and uses it to route into an internal server it could not reach directly.”

website, CyberAgentFlow, What it found

Definition: Using a host you have already compromised as the route to a host you cannot reach from where you started.

Distinguishes it from: Lateral movement in general, which is any spread across hosts at comparable privilege. A pivot names the case where the first host is the only route, so what is acquired is a network path rather than a credential.

Operational test: Cut the first host out of the picture and ask whether the second is still reachable. If it is not, the step was a pivot.

In my project: Chosen as the scenario because it is the point at which tool orchestration stops being optional. Reconnaissance can be done one command at a time by an agent with no memory of the last one. Routing cannot.

observed time against the time without the waste

Definition: The two clocks the analysis compares: what the engagement actually took, and what the same productive steps would have taken with the entries classified as waste deleted from the sequence.

Distinguishes it from: A speedup from a faster model or a faster tool. Nothing in the second number runs faster than it did. It is the same commands in the same order with the repetitions and the uninformative retries removed.

Operational test: Try to reconstruct the second number from the trace by deleting entries and nothing else. If it needs an assumption about what a better agent would have done instead, it is a different measure and a weaker one.

In my project: Roughly fifty minutes against roughly eleven. Neither number is the result. The gap is the result, and the figure on the page is drawn so the gap is the thing you look at.

waste

Definition: Entries in a trace that consumed wall clock and changed nothing about what the agent knew or what the target was.

Distinguishes it from: Failure. A failed exploit that eliminated a module is not waste, it is information bought at a price. Waste is the second attempt at the module the first attempt

already eliminated.

Operational test: Delete the entry from the trace and ask whether any later entry becomes unexplainable. If nothing downstream depended on it, it was waste.

In my project: The classification is made afterwards by an analysis model reading the archived trace, which is exactly why the trace has to hold the outputs and not only the calls. A record of what was invoked cannot tell you whether the answer was already known.

1.4 The five failure patterns

“Five recurring ones, visible only because the trace existed”

website, CyberAgentFlow, The failure patterns

Each of the five is a term. They are not five instances of one problem: four are about orchestration and one is about reasoning, and the page’s conclusion turns on that split.

redundant tool invocation

“the same reconnaissance command run again with unchanged parameters, when the earlier result was still valid”

website, CyberAgentFlow, The failure patterns

Definition: Re-issuing a command whose earlier result had not been invalidated by anything that happened in between.

Distinguishes it from: A deliberate re-scan after the target changed, which is the identical command for a good reason. Redundancy is defined against the validity of the earlier result, never against the repetition itself.

Operational test: Compare the two invocations and everything in the trace between them. If nothing between them could have changed the answer, the second was redundant.

In my project: The pattern that points at the context budget rather than at the model. An agent that still holds the first result does not ask again.

command-context confusion

“commands written for the system shell issued inside Metasploit, or Meterpreter commands issued outside a session. Sometimes hallucinated outright, sometimes correct for a different tool version.”

website, CyberAgentFlow, The failure patterns

Definition: Issuing a valid command to the wrong interpreter, in a setting where several interpreters are reachable through one channel.

Distinguishes it from: A hallucinated command, which no interpreter would have accepted. The page keeps both cases inside one pattern and says so, because from the trace they look identical until you check whether the command exists somewhere.

Operational test: Take the failed command and ask whether it would have worked anywhere. If it would, the fault is the context and not the command.

In my project: The pattern that argues for interfaces rather than for a better model. An agent

that cannot address the wrong interpreter cannot make this mistake, and the fix is on the tool side of the seam.

exploit retry loop

“module after module tried with minimal parameter changes, without folding in what the previous failure revealed”

website, CyberAgentFlow, The failure patterns

Definition: A search that keeps proposing without updating on what its own failures have already ruled out.

Distinguishes it from: Systematic enumeration, which is the same behaviour with a defensible reason: try everything, because none of the failures eliminate anything. A retry loop is enumeration in a setting where the failures did carry information and none of it was used.

Operational test: Read the error from attempt n and ask what attempt $n+1$ did about it. If the answer is nothing, and the error was informative, it is a loop.

In my project: The one of the five that is squarely about reasoning, and the reason the page’s verdict is careful to leave reasoning out of the charge. One out of five is not the profile of an agent that cannot think.

environment constraint violation

“commands assuming shell features or permissions the environment doesn’t offer”

website, CyberAgentFlow, The failure patterns

Definition: Acting on a model of the host that is more permissive than the host.

Distinguishes it from: A failed privilege escalation, which is an attempt to change the constraint and is a legitimate step. A violation is not an attempt on the constraint. It is not having noticed there is one.

Operational test: Ask whether the information needed to avoid the command was already somewhere in the trace. If it was, the agent held it and did not use it, and that is a different repair from teaching it something new.

In my project: This is where the record pays for itself twice, because both the constraint and the command that ignored it are entries in the same file, and the distance between them is measurable.

pivot configuration complexity

“routing requires coordinating Meterpreter session commands with Metasploit modules, and the agent struggles to keep straight which command belongs to which”

website, CyberAgentFlow, The failure patterns

Definition: The failure specific to a task whose steps are split across two tools that have to be driven in alternation, where each individual step is easy and the alternation is not.

Distinguishes it from: Command-context confusion, which is the same symptom with no task-level reason behind it. Here the task genuinely requires two contexts, so the confusion is a property of what was asked rather than an accident of how it was prompted.

Operational test: Ask whether the correct sequence can be written down inside a single tool's command set. If it cannot, the difficulty belongs to the configuration and not to the agent.

In my project: The fifth pattern and the one that names the scenario. It is also where the fifty minutes went: exploit selection, exploit attempts and pivot setup dominated the clock, and pivot setup is this.

1.5 The reading

tool orchestration

“orchestrating the tools it already has may matter as much as choosing them well”

website, CyberAgentFlow, Why this matters

Definition: Driving a fixed set of tools in the right order, in the right context, with each one's output carried into the next.

Distinguishes it from: Tool selection, which is choosing what to reach for. Selection is a judgement about the task. Orchestration is a judgement about the state of the engagement, and it is the one that degrades as the engagement gets long.

Operational test: Hold the tool set and the set of calls fixed and reorder them. If the engagement gets shorter, what was lost was orchestration.

In my project: Four of the five patterns are orchestration failures and one is a reasoning failure. That ratio is the whole argument, and it is the reason the page says the lesson is not that the agent is bad at reasoning.

analysis model

Definition: The model that reads an archived trace after the fact and marks entries as inefficient, distinct from the model that produced the entries.

Distinguishes it from: The agent's own self-critique during the run, which sees only what is still in context and has an interest in the answer. The analysis model sees the complete trace, including the parts the agent's context budget had already discarded.

Operational test: Ask what the marker had access to. If it is the whole archived session rather than the live window, it is the analysis pass.

In my project: It is the component that turns instrumentation into a finding. The trace makes waste visible in principle; the analysis pass is what makes reading fifty minutes of it tractable.

success rate

“you cannot see any of this from a success rate”

website, CyberAgentFlow, Why this matters

Definition: The fraction of runs in which the objective was reached, and the single number most published evaluations of agentic cyber tools report.

Distinguishes it from: Any measure of cost. The two are independent, and this engagement is the demonstration: it scores one out of one and spent four fifths of its clock on nothing.

Operational test: Ask what the metric would report about two runs that both succeeded, one in eleven minutes and one in fifty. If the answer is the same number twice, it is a success rate.

In my project: The charge is not that the number is wrong. It is that the number is thin, and that the field currently has almost nothing else. Reported end-to-end success rates for penetration testing agents sit around a third, which makes the failures easy to count and the successes impossible to inspect.

single run

“Observations come from a single run. They illustrate common failure modes rather than establishing their frequency.”

website, CyberAgentFlow, the note under the finding

Definition: The scope statement attached to the result: one engagement, five patterns named, no claim made about how often any of them occurs.

Distinguishes it from: A pilot, which is a small version of a study that will later run at scale in the same design. This is a demonstration that the instrument records what it says it records, and the five names are its output.

Operational test: Ask what a second run could overturn. It could change every frequency and none of the five names.

In my project: The sentence sits in italics on the page under the finding, and it decides what the extended abstract is allowed to claim. It is also the sentence that makes the next version of this project a frequency study rather than a second demonstration.

2 Working

Metasploit and Meterpreter

The two tools whose relationship produces two of the five patterns. Metasploit is the framework holding exploit modules and their parameters, with its own console and its own command vocabulary. Meterpreter is the payload that runs on a compromised host and gives a session with a third vocabulary. Routing a pivot means alternating between them, which is why the agent has to hold in mind which of three command sets it is currently speaking.

nmap and service identification

The reconnaissance step and its tool. Scanning enumerates reachable hosts and open ports; service identification says what is listening and at what version, which is the input exploit selection consumes. Both are cheap, both are idempotent while the target holds still, and that is what makes a repeated scan legible as waste.

Ollama and LiteLLM

The two components that make the model private without making it awkward. Ollama runs the model on local hardware. LiteLLM presents whatever is running behind one calling convention and applies access control on the way through. Neither is the contribution; both are the reason the local stack is a practical position rather than a stated preference.

Kali

The distribution the tools come from, and the reason the tool set is fixed rather than designed. The agent is given what a human tester would already have on the machine, which is what makes the orchestration finding transferable.

objective in natural language

How the engagement starts. The analyst states a goal in a sentence rather than supplying a plan or a target list, so the decomposition into steps is the agent's work and is therefore in the trace. If the analyst supplied the steps there would be no orchestration left to measure.

observability

The word the published abstract uses for what the framework improves. It names the property of a system whose internal state can be reconstructed from what it emits. The five patterns are the argument that an agent driving tools is not observable by default, because everything that distinguishes them from each other happens between the prompt and the command.

3 Peripheral

penetration testing agent benchmarks

The evaluation line this project is arguing with rather than joining. Benchmarks assemble vulnerable targets and score whether an agent reached the flag, and the published systems cluster in the low thirties as a percentage. They answer whether the field is progressing. They cannot answer what any single run cost.

trace semantic conventions

The vocabulary the general agent-tooling world converged on for the same object: a span per model call and per tool invocation, with the model, the arguments, the token counts, the latency and the outcome as attributes, linked by a correlation identifier. It is the same idea as an execution trace arrived at from operations rather than from research, and it matters here because it fixes what a domain-agnostic version of this record would look like.

agentic end point defense

The defensive counterpart on my Headspace page, where the question is whether a defender agent can be prototyped and evaluated at all. The instrumentation argument transfers unchanged. A defender agent that reports only whether the network survived is as opaque as an attacker agent that reports only whether it got in.

4 The disagreements, listed

The disagreement	The entry it sits in
Four spellings on the site, and a fifth scope in the paper	CyberAgentFlow
Trace as a team record, trace as one agent's tool use	execution trace
Authority as a boundary set in advance, authority as an approval routed during	tool permission and IP allowlist
Complexity as a property of the task, confusion as a property of the agent	pivot configuration complexity

Two things called a trace

CHART records the linked Input-Process-Outcome history of a human-AI team session. Its unit is a coordination sequence across several teammates, and a failure in it is attributed to a structure: an approval that routed wrong, an explanation that never arrived.

CyberAgentFlow records the execution history of one agent driving real tools. Its unit is a prompt, a tool invocation and its output, and a failure in it is attributed to a wasted call.

Both are traces under the shared definition, which is a time-stamped record whose entries are linked to one another. They differ in what the linked entities are, and therefore in what a failure can be attributed to. The cluster overview page uses the bare word for both in a single sentence. Any file that has to be precise says team trace or execution trace, and this file says execution trace throughout.

The authority disagreement is the one worth stating twice, because it looks like an inconsistency and is not. CHART externalizes the authority mechanism so that approving, denying and the delay before either becomes measurable. CyberAgentFlow sets the boundary before the engagement starts and records nothing about it, because a live engagement against a real host is not a place to study a person's approval behaviour. Two projects in one cluster with opposite designs, for stated reasons. The vocabulary should keep them apart rather than reconcile them.

5 What crosses into the shared lexicon

Terms this project uses that the whole Human-AI Complementarity cluster uses. Their cluster sense sits in the shared file, and the entries above carry only what CyberAgentFlow fixes about them: trace, evidence, authority, human in the loop, and the cluster's second challenge about attributing a failure rather than inferring it.

Four terms are fixed here and nowhere else in the cluster. Execution trace is the only record in my files whose unit is a single agent's tool call. Waste is the only quantity in my files that is defined by deletion rather than by measurement. The five failure patterns are the only named failure taxonomy anywhere in the cluster, and they are named at the level of the agent's behaviour rather than at the level of the team's structure. And the context budget is the only term in the cluster that is about a model's memory rather than about a team's memory.

One term comes in from outside and should stay attached to its source. Success rate is the metric

the whole Toward Deployment cluster is also arguing with, on different grounds: there the complaint is that it does not survive a change of environment, and here the complaint is that it does not survive a stopwatch.

6 Sources

- Every quoted sentence is from the CyberAgentFlow page of the website. The locator on each entry says which section of it.
- The four spellings are from the page file name, the page heading, the page body and `content/SUMMARY.md`.
- The framework's domain-general name, its stated aim of improving observability, the six recorded entry kinds and the three summary categories of inefficiency are from the published extended abstract, Cyber-Agent-Flow, AAAI Symposium Series 9(1), 337 to 340, by J. Acosta, M. T. B. Nazim, T. Guerra, Y. Du and P. Aggarwal.
- The requirement that a trace be detailed enough to attribute a failure to the structure, and the one-line description of this project alongside the other three, are from the Human-AI Complementarity overview page. The competing sense of trace is from `chart.md` and from `content/artifacts/chart/design.md`, and belongs properly to `hac_chart.tex`.
- The reported success rates for penetration testing agents are from the current benchmark literature, which puts end-to-end completion for the published systems around a third.
- The span vocabulary in the peripheral tier is the generative AI semantic convention set, which fixes model, arguments, token counts, latency and outcome as span attributes.
- The three tiers, the three fluency levels and the four fields are from the Tech Fluency page. The reason a term whose two uses disagree belongs inside its entry is in `lexicon_method.tex`, in this folder.